



윤예진 프론트엔드 개발자

빠르게 기능을 구현하고 배포해 실제 사용자 반응을 확인하며 제품을 다듬는 과정에 가장 큰 흥미가 있습니다.

개발 과정의 비효율을 찾아 자동화·표준화로 문제를 해결하고, 사용자의 불편을 관찰하여 서비스화 한 경험이 있습니다.

주도적으로 목표를 재정의하고 빠르게 학습하고 적용하는 태도를 갖추어 팀에 더 큰 가치를 제공할 수 있도록 노력하겠습니다.

Email.

yyj11kr@naver.com

GitHub.

<https://github.com/Yejin000>

| shadcn-dialog-snippet

기간: 2024.09 ~ 2025.03

오픈 소스 - <https://shadcn-dialog-snippet.vercel.app> | <https://github.com/Yejin000/shadcn-dialog-snippet>

기술스택: React, TypeScript, Vite, Tailwind CSS, Mixpanel, shadcn/ui, Biome

shadcn/ui의 Dialog 컴포넌트를 다양한 사례와 함께 제공하고자 shadcn-dialog-snippet을 기획했습니다. 개발 과정에서 발생한 비효율을 개선하고, 개발 방향성을 설정하기 위한 작업을 진행했습니다.

개발자 경험(DX) 개선

컴포넌트 코드를 문자열로 변환해 사용하는 과정에서 수동 스크립트 실행으로 인해 변환 누락과 반복 작업이 발생했습니다. 이를 해결하기 위해 Vite 플러그인을 작성하여 개발 서버 실행 시 buildStart 훅에서 전체 파일을 변환하고, 파일 저장 시 handleHotUpdate 훅으로 수정된 파일만 다시 처리하도록 구성했습니다.

이 방식으로 변환 과정을 개발 서버와 연동해 별도 명령 없이도 항상 최신 문자열이 유지되도록 했습니다.

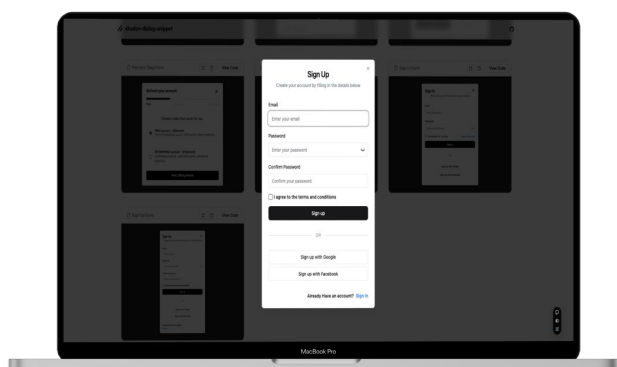
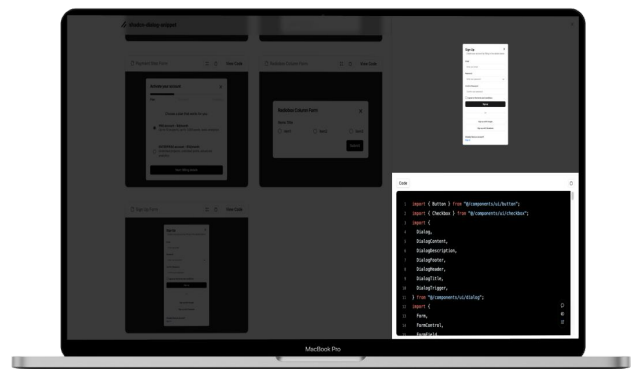
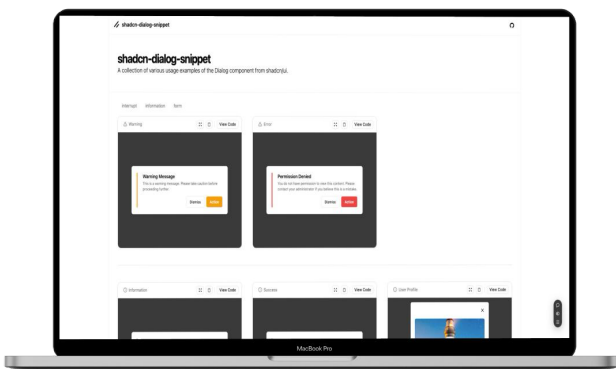
- `**/*Base.tsx` 패턴의 파일을 대상으로, TSX 코드를 문자열로 변환하는 Vite 플러그인을 구현
- buildStart 훅으로 서버 실행 시 전체 파일 변환, handleHotUpdate 훅으로 수정된 파일만 반영해 HMR 지원
- 기존 변환 결과와 비교 후 변경된 경우에만 갱신하고, 변환 후 biome으로 자동 포맷 처리

사용자 행동 데이터 추적 위한 파이프라인 구축

사용자가 어떤 다이얼로그 유형을 가장 많이 참고하는지 파악하고, UI 개선, 콘텐츠 최적화, 향후 개발 방향성을 설정하기 위해 클릭 패턴을 분석해야 했습니다.

카테고리 바로가기 버튼에 Mixpanel 이벤트 트래커를 적용하여 클릭 데이터를 수집하였습니다.

- 카테고리별 클릭 이벤트를 추적하여, 사용자가 어떤 다이얼로그 유형을 가장 많이 참고하는지 분석할 수 있는 환경을 구축
- 클릭 데이터를 기반으로 UI 및 콘텐츠 배치를 최적화할 가능성을 열어둠
- 사용자 행동 데이터를 수집하여, 향후 개발 방향성을 구체화할 기반을 마련



배드민턴 동호회 서비스 - https://github.com/Kernel360/F2_PLAY_BADMINTON_FE

기술스택: Next.js, TypeScript, Tanstack Query, Tailwind CSS, shadcn/ui, Storybook, Biome

배드민턴 동호회 플랫폼 개발 과정에서, 사용자 불편을 줄이고 필요를 반영하기 위해 적절한 기술을 선택하고 이를 서비스에 녹여내는 데 집중했습니다. 실시간 데이터 처리와 직관적인 UI 설계, 검색 엔진 최적화를 통해 사용자 경험을 개선하려 노력했습니다.

사용자 경험 개선을 위한 optimistic update 적용

경기 점수 입력과 업데이트 과정에서 발생하는 지연과 혼란을 줄이기 위해 **debounce**와 **optimistic update**를 결합했습니다. debounce를 통해 서버와의 불필요한 통신을 최소화하고, optimistic update를 적용하여 점수 변경 사항이 서버 응답 이전에 UI에 즉시 반영되도록 구현했습니다.

서버 컴포넌트로 SEO 강화

동호회 홍보와 신규 사용자의 접근성을 높이기 위해 검색 엔진 최적화를 진행했습니다. 서버컴포넌트를 활용하여 서버에서 **prefetch + dehydrate**로 초기 캐시를 전달하였고, 서버 사이드 렌더링(SSR)을 진행했습니다. 동적 사이트맵을 도입했고, 동적으로 메타 태그도 생성하였습니다.

에러 핸들링을 위한 공용 hook 제작

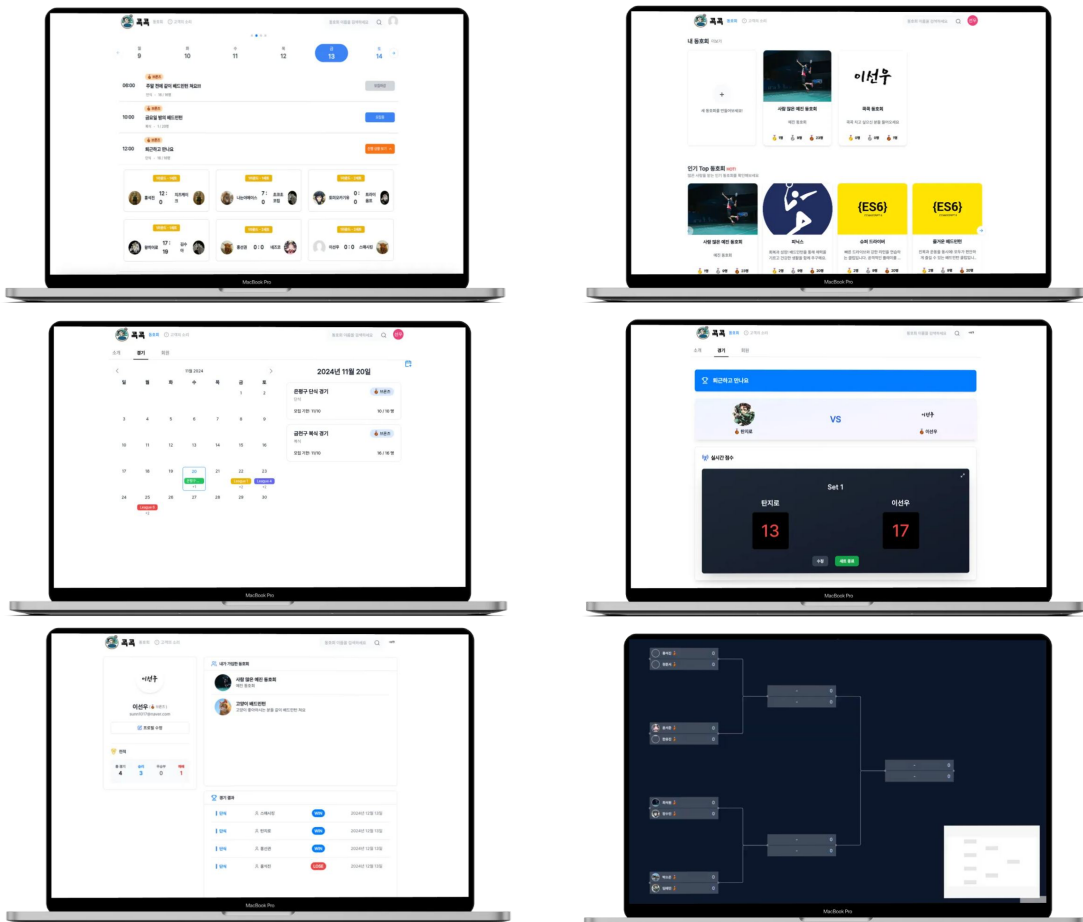
데이터 요청과 전송 과정에서 발생할 수 있는 에러를 관리하기 위해 데이터 처리 로직을 하나의 공용 Hook으로 통합했습니다. 에러 발생 시 Toast 알림으로 문제를 전달하고, 성공적인 요청은 사용자 맞춤 동작을 실행할 수 있도록 설계했습니다.

대진표 시스템 설계

토너먼트 경기 진행 상황을 시각화하고 실시간 업데이트를 제공하는 기능을 구현했습니다. 토너먼트 구조를 시각적으로 표현할 수 있는 시스템을 도입하여 경기 결과와 대진 흐름을 사용자들이 한 눈에 볼 수 있게 되었습니다. react-flow 라이브러리를 활용하여 노드와 엣지 기반의 시각화를 구현했습니다. 대진표의 경기 연결 관계를 표현하고, 각 경기 정보를 노드에 추가하였습니다.

디자인 시스템 설계

디자인 토큰을 정의하여 Primitive 컴포넌트의 임의 스타일 사용을 제한하여 토큰만으로 렌더되도록 통일했습니다. Storybook에서 Docs/Controls로 컴포넌트의 변형과 상태, 포커스 흐름을 토큰 값 기준으로 명세하여 페이지 구현 전에 모두가 같은 스펙을 참조하게 했습니다



IBAS 기간: 2021.02 ~ 2024.07

동아리 LMS 사이트 - <https://www.inhabas.com> | <https://github.com/InhaBas/inhabas.com-front>

기술스택: React, TypeScript, Vite, Styled Components, Recoil

100명 규모 동아리의 회원 관리 및 활동 조율 문제를 해결하기 위해 **Django**를 기반으로 **LMS사이트**를 만들었습니다.

Django는 웹 애플리케이션 개발에 필요한 기본 기능을 제공하여 빠르게 시스템을 구축할 수 있었지만, 유지보수성과 재사용성 문제, 동적 인터랙션 구현의 한계에 직면하면서 React로의 마이그레이션이 필요했습니다.

React로의 마이그레이션

서버 로직과 클라이언트 로직이 결합되어 있어 작은 수정에도 전체 코드를 검토해야 했으며, 동일한 UI 요소를 반복적으로 작성해야 하는 구조로 인해 코드 중복이 발생했습니다. 이를 개선하기 위해 React로 전환하면서 컴포넌트 기반 설계를 도입했고, 변경 사항이 생겨도 최소한의 수정만으로 유지보수가 가능하도록 구조를 개선했습니다.

동적 인터랙션 개선

Django 기반의 서버 렌더링 방식에서는 특정 버튼 클릭 시 일부 콘텐츠만 업데이트하는 것이 어려웠고, 페이지 전체를 새로고침해야 하는 문제가 있었습니다. React로 전환하면서 클라이언트 측에서 Recoil이나 useState로 상태를 관리하고 변경 사항을 즉시 반영할 수 있도록 했으며, 사용자 경험을 향상시켰습니다

권한 체계 간소화 및 UI 재설계

기존 시스템에서는 18개의 권한 체계를 운영했으나, 관리 복잡성을 줄이기 위해 9개로 축소했습니다. 권한별 UI와 접근 로직을 재설계하면서 기능 충돌 가능성과 사용자 경험 변화를 검토했습니다. 기존 코드의 의도를 정확히 이해하는 것이 중요했기 때문에, 권한 로직과 데이터 흐름을 단계별로 신중하게 검토하며 전환을 진행했습니다

